



INDIVIDUAL ASSIGNMENT 1

20%

SUBJECT NAME

BIT1123 Object-Oriented Programming (Java)

No.	Name	Student ID	Programme
1.	GHAITH AL-DAIN GAMIL QASEM	202501010660	BCSSE

Class Code	202605F0782
Program	BCSSE
NRIC/Passport Number	11772197
Report	Self-Reflective Report: Object-Oriented Programming Fundamentals in Java

SUBMITTED TO: SIR NAZMIRUL IZZAD BIN NASSIR

1. Introduction

This report reflects on my learning experience across the Java tutorials completed for BIT1123 Object-Oriented Programming. At the beginning, I was mainly focused on understanding Java syntax and getting small programs to run. As the tutorials progressed, I started working with several classes and thinking about how objects should share or protect data. The repository records that progress from a basic Hello World program to a graphical quiz application. It also shows how each topic prepared me for the next one.

2. Knowledge gained from each tutorial

Week 1: Java fundamentals

In Week 1, I learned the basic structure of a Java program, including the class declaration and the main method. HelloWorld.java gave me a simple starting point for compiling and running code. In StudentGrade.java, I worked with variables, data types, if-else statements, and console output. The grading conditions helped me see how a program follows different paths depending on a value.

Week 2: Classes and objects

Week 2 introduced classes and objects through a Student class. I defined attributes for a student's name, age, and GPA, then used a constructor to set those values. I also added methods for displaying information, studying, and taking an examination. Creating a Student object in the Main class made the difference between a class definition and an object much clearer to me.

Weeks 3-4: Inheritance and polymorphism

The work for Weeks 3 and 4 used Person as a parent class, with Student and Lecturer as child classes. I learned how extends creates an inheritance relationship and how super passes values to the parent constructor. Each child class overrides the introduce() method. When Person references hold different object types, the correct version of introduce() runs for each object. This example helped me understand polymorphism as behaviour that depends on the actual object, not only the reference type.

Week 5: Encapsulation

In Week 5, the Student fields were private and could only be accessed through getters and setters. This showed me why a class should control access to its own data. Instead of changing a field directly from the main program, I used methods such as setCGPA() and getCGPA(). Encapsulation made the class easier to manage and provides a place to add validation later if the program becomes more complex.

Week 6: Inheritance with constructors

Week 6 continued inheritance with Employee and Lecturer. General details such as the ID and name belong to Employee, while subject and department belong to Lecturer. The Lecturer constructor calls super() before assigning its own fields. This exercise helped me avoid repeating common attributes and methods in related classes.

Week 7: Abstraction

Week 7 introduced an abstract Appliances class. It contains shared behaviour for displaying the brand and controlling power, while `operate()` is left abstract. `WashingMachine` and `Refrigerator` implement that method differently. I learned that abstraction can define what related classes must do without forcing them to perform the action in the same way.

Weeks 8-9: Collections and file handling

For Weeks 8 and 9, I created a small task manager using `Scanner` and `ArrayList`. The program collects three tasks, displays them, writes them to `task.txt`, and reads them back. `BufferedWriter`, `BufferedReader`, `FileWriter`, and `FileReader` showed me how a program can preserve information outside its running session. The try-catch blocks also introduced a practical reason for exception handling because file operations can fail.

Week 10: Java Swing GUI

The final tutorial moved from console programs to a graphical interface. `QuizBattleGUI` uses `JFrame`, `JLabel`, `JButton`, fonts, and an `ActionListener`. The `Questions` class stores the question, options, and correct answer, while the GUI handles what the user sees and clicks. Separating these responsibilities made the project easier to follow. Event handling was new to me because the program waits for a button click instead of running only from top to bottom.

3. Challenges encountered

One challenge was keeping track of relationships between classes. In the inheritance tutorials, I had to understand which fields and methods belonged to the parent and which belonged to the child. Method overriding was also confusing at first because several classes used the same method name but produced different output. File handling presented another difficulty because streams must be opened, used, and closed correctly, and an `IOException` may occur. In the Swing tutorial, arranging components and connecting buttons to the `ActionListener` required more attention than a console program.

4. How I overcame the challenges

I handled these problems by working through each program in small parts. For inheritance, I followed the constructor calls from the child class back to the parent and tested each overridden method separately. For file handling, I traced the program in order: collect the tasks, save them, close the writer, open the reader, and print each line. In the GUI project, I separated the question data from the interface and checked the result after each component or event was added. Compiling and running the programs regularly helped me locate errors before they became harder to identify.

5. Improvements in my Java programming skills

My Java skills improved in both syntax and program structure. I can now create classes with constructors and methods instead of placing all logic in the main method. I am more comfortable using access modifiers, getters and setters, inheritance, abstract classes, ArrayList, file streams, exception handling, and Swing components. I also became more careful about file names, class names, and the way several source files work together in one project.

6. Understanding of object-oriented programming concepts

The tutorials gave me a practical understanding of the main object-oriented programming concepts. Encapsulation protects an object's data and exposes controlled methods for using it. Inheritance allows a child class to reuse and extend a parent class. Polymorphism allows the same method call to produce behaviour that matches the actual object. Abstraction defines common behaviour while leaving specific details to subclasses. I now see these concepts as connected design tools rather than separate definitions to memorize.

7. Future learning plans

My next goal is to practise larger Java programs that combine these topics instead of demonstrating them one at a time. I want to improve input validation, learn more about interfaces and exception design, and work with collections that store objects rather than only strings. I also plan to study database connectivity and improve my Swing layouts so that future applications can store real data and provide a better user experience.

8. Conclusion

This assignment helped me review the progress recorded in my tutorials. The early exercises established the basic Java syntax, and the later work showed how object-oriented design can organise a program into related classes with clear responsibilities. Collections, file handling, and Swing also showed how Java programs can manage information and respond to users. I still need more practice with larger applications, but I now have a stronger foundation for planning, writing, and testing object-oriented Java code.

9. GitHub repository URL

https://github.com/8voux/Ghaith-Al-Dain-Gamil-Qasem_-202501010660_-Java